

OkBuddy AI-Assisted Content Generation – Work Experience & Summary Product Specs

Work Experience Section – AI-Assisted Content Generation

Problem Statement

Users report that the current “Tạo nhanh” (Quick Generate) feature for work experience often produces irrelevant or generic bullet points that don’t match their actual work. This leads to frustration and distrust in the AI suggestions – many feel the generated content is unusable or requires heavy editing. The core issue is a **mismatch between AI output and the user’s real experience**, undermining the goal of helping users overcome writer’s block.

Root Cause Analysis

- **Lack of Context:** The one-click generation today relies only on minimal inputs (e.g. job title) to produce bullets. Without specific context or achievements from the user, the LLM falls back on generic responsibilities. As a result, suggestions feel impersonal and often inaccurate for the user’s role.
- **User Input Omission:** Users currently aren’t prompted to provide any details. The AI is essentially guessing the user’s accomplishments. This “blind” generation yields low relevance, as concept testing showed. Users feel **“the AI doesn’t understand what I actually did”** – a direct outcome of not gathering user context.
- **Perceived Loss of Control:** Immediately injecting AI-generated bullets makes users feel the content is imposed on them. They have no say until after the fact, which can be demotivating. The lack of a guiding process means the AI’s contribution overshoots or misses the mark, rather than working with the user.

Solution Overview

We propose a new AI generation flow that keeps users in the loop via a lightweight interactive wizard and fallback scaffolding. Instead of one-shot generation, the **AI will collaborate with the user to produce tailored bullet points**. The solution has three components working together:

1. **Contextual Prompting Wizard:** When the user clicks “Tạo nhanh” for a job, we initiate a 2–3 step micro-wizard. It asks the user targeted questions about their experience in that role (e.g. a key project, the impact/result, and their responsibility). These answers provide the LLM with personalized context to generate a highly relevant bullet point.
2. **Skeleton Template + Smart Autocomplete (Fallback):** If the user opts to skip the wizard or struggles to answer, the system offers structured bullet templates (e.g. “Led [team/task] to [outcome], resulting in [impact].”). The user can fill in the blanks with their specifics. As they type, an AI-powered autocomplete may suggest completions to help finish the sentence. Once the blanks are filled, the LLM can polish the phrasing into a well-crafted bullet.
3. **“Cải thiện với OkBuddy AI” (Improve with AI) CTA (retained):** For any bullet point the user has written (manually or via wizard), a one-click “Improve” button remains available. This uses the LLM to rewrite the bullet for clarity, impact, and ATS-friendly keywords, while preserving the original meaning. It’s an instant optimizer that refines wording and adds quantifiable outcomes if possible.

This new approach ensures the AI’s output is grounded in **user-provided data**, maintaining the benefits of beating the blank page and high-quality examples, without diverging from the user’s actual experience.

New UX Flow (Step-by-Step)

1. **Triggering the Wizard:** In the Work Experience section, the “Tạo nhanh” button (currently a lightning icon ⚡ on the job entry or similar) now opens an **interactive wizard modal** instead of immediately generating content. The modal is lightweight – a small dialog or popover – to keep the user in context of their resume editing.
2. **Wizard Step 1 – Key Project/Task:** The wizard’s first prompt asks, **“What was one key project or task you accomplished in this role?”** The UI presents a single-line text field (with a short placeholder example like *“e.g. developed a new customer CRM system”*). The user inputs a brief description of a significant project or responsibility. A “Next” button becomes enabled once text is entered.
3. **Wizard Step 2 – Impact/Result:** The second prompt asks, **“What was the impact or result of that work?”** Again a text field is provided (placeholder: *“e.g. improved data retrieval speed by 30%”*). The user describes the outcome or benefit of the project. (If the user doesn’t have a quantifiable metric, they can write a qualitative result, like *“streamlined the sales process”*). They click “Next”.
4. **Wizard Step 3 – Primary Responsibility (Optional):** The wizard asks, **“What was your primary responsibility in this role (overall)?”** This step is optional and meant to

capture context like team leadership or scope (*“e.g. led a team of 5 engineers”*). If the user’s input in step 1 already implies this, they may skip. The wizard UI shows a **“Skip”** option for this step (and in any step, a Skip is allowed to not force an answer).

5. **Wizard Completion – Generate Bullet:** The final screen has a **“Generate Bullet”** button. It may be enabled only after at least the first two questions have some input (project and impact are the minimum for a meaningful result). When clicked, the wizard dialog calls the LLM with all collected inputs. A loading spinner or progress indicator appears on the button (and possibly a subtle *“OkBuddy is writing your bullet...”* message) to indicate work.
6. **Bullet Insertion:** Upon completion, the wizard closes and a new bullet point is created into the job’s description field list. The generated text is based on the user’s inputs, combined into a polished accomplishment statement. For example, if the user entered *“developed a new CRM system”* and *“improved data accuracy 30%”*, the bullet reads: *“Led development of a new CRM system, improving data accuracy by 30%.”* The bullet appears highlighted to indicate it’s AI-suggested and new. The right-side preview updates immediately to reflect this addition.
7. **User Review & Edit:** The user can review the bullet. They remain in full control – they can edit the text inline to tweak specifics or tone. The AI-proposed bullet is intended as a high-quality first draft that the user can adapt, mitigating the blank page syndrome while staying relevant.
8. **Repeat for Additional Bullets:** The wizard can be re-invoked for the same job to generate another bullet. For instance, after finishing one bullet, the user might click *“Tạo nhanh”* again (the button now says *“Add another bullet with AI”*). The flow repeats, prompting for another project/task and result. Each run produces one distinct bullet focusing on a different accomplishment. In this way, a user can iteratively build out multiple bullet points by cycling through the wizard for each major achievement they want to highlight.
9. **Context Preservation:** The wizard is always aware of which job entry it was launched under. It can display the job title/company at the top of the dialog for context (e.g., **“Software Engineer @ ABC Corp – Generate bullet”**). This reminds the user which role they are adding details to, and also allows the prompt to include the job title internally for the AI’s context (helping it tailor the tone or jargon if needed).
10. **Exit or Cancel:** The user can cancel the wizard at any time (e.g., via an *“X”* close button or a *“Cancel”* link). If canceled, no AI call is made and nothing is inserted. The job description remains as it was. This gives users an out if they change their mind or realize they’d prefer to write manually.

Fallback States – Skeleton Templates & Autocomplete

Not every user will have an easy answer for the wizard questions, and some may skip it. To support them, we provide a fallback that still prevents the dreaded blank page:

- **Skip Wizard Path:** If the user clicks “Skip” during the wizard (or cancels it), the system will present **editable bullet templates** to jumpstart writing. For example, upon skipping, a dropdown or small list appears in the job entry area titled “**Bullet Templates:**”. It contains a few common resume bullet structures like:

- “Led [team/task] to [outcome], resulting in [impact].”
- “Implemented [project/initiative] that [result achievement].”
- “Improved [process or metric] by [X%] through [specific action].”

Each template is shown with placeholder slots (indicated by brackets or light-highlight text).

- **Inserting a Template:** The user can click on one of the template options. Upon selection, a new bullet text field is created in the editor pre-filled with that skeleton structure. For instance, clicking the first template puts a bullet that literally reads: “Led [team/task] to [outcome], resulting in [impact].” inside the Work Experience entry, with the placeholders highlighted.
- **Filling the Blanks:** The user replaces each placeholder with their information. The placeholders can be interactive tokens – clicking a [team/task] slot focuses it for typing, and it disappears as the user types. As an example, the user might fill: “Led **a team of 5 engineers** to **deploy a new CRM**, resulting in **30% faster data retrieval**.”
- **Smart Autocomplete Assistance:** While the user is typing into a skeleton, the AI provides real-time suggestions to help complete the sentence. For instance, as the user types the action or outcome, a greyed-out continuation might appear. If the user pauses after typing “resulting in”, the system could suggest “*a 30% increase in sales*” based on context or common results, which the user can accept or ignore. This GPT-powered autocomplete (akin to a “Smart Compose” feature) reduces the effort to come up with phrasing. It leverages partial user input to predict the rest, offering a quick way to fill in details the user hasn’t typed yet.
- **LLM Completion (Polish):** Once the user has filled the blanks to the best of their ability, they can invoke the AI to “**Complete & Refine**” the bullet. This could be a small “✨ **Complete with AI**” button appearing next to the field (especially if any placeholder remains or the sentence is fragmented). Clicking it sends the current bullet text (with user-provided fragments) to the LLM, which then returns a polished, grammatically

correct sentence. It essentially replaces any remaining placeholders or fixes awkward phrasing. For example, if the user only filled “Led a team of 5 engineers to deploy a new CRM” and left the impact blank, the AI might complete it as: “Led a team of 5 engineers to deploy a new CRM, improving data retrieval speed by 30%.” (The AI can infer a generic positive impact if the user skipped one, though it’s best with actual data).

- **No Empty Bullet:** If the user inserts a skeleton but doesn’t fill any info and tries to finalize, the system will not generate a meaningful bullet. In this case, we keep the bullet field empty or guide the user to input something. Essentially, the skeleton is a prompt for user input; without any, we don’t call AI. We might display an inline tip like “Fill in at least one detail to generate a bullet point” if they attempt to use AI complete too early.
- **Template Variety:** Multiple templates ensure users in different scenarios have a relevant structure. For instance, a sales role might prefer the “Improved [metric] by [X] through [action]” format to highlight numbers, whereas a leadership role might use “Led [team] to [achieve outcome]”. By providing a couple of choices, we address various work experience styles. This library can be expanded over time or even filtered by the job title (e.g., if the user’s title is “Manager”, show templates emphasizing leadership) .
- **Reverting to Wizard:** The fallback doesn’t eliminate the wizard option. A user can still decide to use the wizard after seeing templates, if they realize they do have specifics to input. Thus, the “Tạo nhanh” entry point could offer both: e.g., in the dropdown under the AI icon, list “Guided Bullet Wizard...” and “Use a Bullet Template...”. But in the default flow we assume wizard first, template as secondary if skipped.

AI Prompt Logic for Bullet Generation & Improvement

The AI prompting strategy is designed to use **user-supplied context** to produce succinct, achievement-oriented bullet points. Below are the key prompt formulations:

- **Wizard-Driven Generation Prompt:** When the user submits the wizard, the system constructs a prompt for the LLM that includes the relevant facts:
 - The **job title and company** (from the Work Experience entry the user is editing) to set context (e.g., *“Role: Software Engineer at ABC Corp”*).
 - The **user’s answers** for Project/Task, Impact, and Responsibility. For example: *“Project: developed a new internal CRM system; Result: improved data accuracy by 30%; Responsibility: led a team of 5 engineers.”*
 - An **instruction** to the AI to compose a single bullet point that combines these elements. The instruction emphasizes resume style: start with a strong action verb, focus on the achievement (result), be concise, and use quantitative metrics

(like that 30%) . The tone should remain professional and in third-person (no “I” pronouns).

- *Prompt Template Example: “You are an expert resume writer. Using the details given, create one bullet point describing the user’s accomplishment in this role.\nRole: Software Engineer\nKey Project: Developed a new internal CRM system\nOutcome: Improved data accuracy by 30%\nPrimary Responsibility: Led a team of 5 engineers\n\nBullet:”* – The model will output something like *“Led a team of 5 engineers to develop a new internal CRM system, improving data accuracy by 30%.”*
- If the **Responsibility** field was provided, the AI can choose to integrate it (as it did by leading with “Led a team...”). If it wasn’t, the AI might just focus on project and outcome: *“Developed a new internal CRM system that improved data accuracy by 30%.”* The prompt logic accounts for missing pieces by excluding empty fields from the final formatted prompt, so the model isn’t confused by a blank. It will still produce a coherent sentence with what it has.
- **Skeleton Completion Prompt:** When the user fills a template partially and clicks “Complete & Refine,” the prompt to the LLM includes:
 - The **partial bullet text** as the user has written it. For instance: *“Led a team of 5 engineers to deploy a new CRM system, resulting in”* (user stopped at “resulting in”).
 - An instruction to **finish and polish** the sentence. E.g., *“Continue and refine the bullet point in a professional resume tone. Ensure the sentence is complete and highlights a result or impact.”*
 - The model will then produce the completed sentence, e.g., *“...resulting in a 30% increase in data retrieval speed.”*
 - If the user’s partial input already forms a complete sentence but perhaps is awkward, the instruction might be to rewrite it more strongly. In either case, the LLM’s task is constrained to **enhancing or finishing the given text**, not inventing unrelated content. This keeps the output aligned with what the user provided. It effectively mirrors the one-click “Improve” functionality but for an incomplete draft.
- **One-Click “Improve Bullet” Prompt:** The “Cải thiện với OkBuddy AI” option for a bullet uses a refined version of our current prompt:
 - It sends the **existing bullet text** to the LLM with instructions to **rephrase it for clarity, impact, and keyword optimization**. We specifically include guidelines

to:

- Use a stronger action verb if possible.
 - Make the sentence more concise and active (avoiding passive voice or fluff).
 - **Quantify the achievement** if the content implies a measurable outcome (e.g., if the bullet says “improved user engagement”, the AI might add “by 15%” if it infers or knows a typical range – though it won’t fabricate a number out of thin air; it might instead suggest adding “[number]” as a placeholder).
 - Ensure any industry or role-specific **keywords** (especially those from a target job description, if available) are included naturally .
- The tone instruction is to maintain professional resume language and keep the point in third person past-tense. For example, if the user wrote “I was responsible for managing the team’s sprint planning”, the AI might output “Managed team sprint planning to ensure on-time delivery of features.”
 - The improved result is returned and replaces the bullet text in the editor (with change tracking or highlight). The user can accept or revert it. *This mirrors the current behavior, but with updated prompt tuning to prioritize quantified outcomes and ATS keywords.*
 - We allow **regeneration**: if the user clicks “Improve” again on the same bullet, we can call the AI for an alternative phrasing. This cycle can yield multiple options (perhaps cycling through synonyms or formats) until the user is satisfied .
- **AI Parameters & Constraints:** All prompts will specify a limit on length (e.g., “keep the bullet under 30 words”). The model is instructed not to introduce new information not given by user (except minor logical assumptions like timeframe or generic outcomes if absolutely needed). We use a tone that is factual and achievement-oriented. We also bias the AI to preserve any **numbers or specifics** the user provided – those should appear in the output unchanged unless the transformation is purely grammatical. This ensures e.g. a “30%” stays as “30%” and not “thirty percent” or lost.
 - **Prompt Variation – Missing Inputs:** For cases where a user skips an input:
 - If **Outcome/Result** is missing, the AI prompt will still ask for an achievement-oriented bullet, but acknowledges that no specific metric was given. The instruction might be, *“If no quantitative result is provided, describe a qualitative impact.”* The AI might then produce a sentence focusing on the

improvement qualitatively, e.g., “Led development of a new CRM system to streamline data management processes.” Additionally, we might append a gentle suggestion in the UI afterward: *“Consider adding a specific result for even stronger impact.”* (This ties in with potential metric suggestions discussed in edge cases).

- If the **Project/Task detail** is missing (user skipped the first question – unlikely since it’s essential), the AI would default to a generic description based on job title. However, we actually will **prevent this scenario** by guiding the user that the first question is required (we could disable “Next” until something is entered). In the rare case we proceed with only an outcome and no description, the bullet might come out too vague (“Achieved a 30% improvement in X through initiatives”), which is not ideal. Thus, the design encourages at least some project detail.
- If the user skips the **Responsibility** input, it’s not critical – the prompt simply won’t include it, and the AI will generate a bullet just fine (it might focus solely on the project and result, which is acceptable). This field is supplementary context, so its absence is handled gracefully by omission.
- In the skeleton fallback, if the user only fills part of a template then hits “Complete with AI,” the prompt will contain exactly what they typed. If that’s insufficient for a meaningful bullet (e.g. they only entered an outcome but not what they did), the AI might produce a generic sentence like *“Achieved a 15% increase in sales through unspecified initiatives.”* To avoid nonsense, we again encourage the user to fill at least the action part of the template. The completion function may refuse to run if the main action part is empty.
- **Missing Job Title context:** In rare cases the user might not have entered a Job Title or Company for the work entry. Our prompt can still work without it, but the generation might be generic. We will check: if job title is blank, we might prompt the user to fill it for better results (since a “Software Engineer” and “Marketing Manager” would have very different bullet styles). This is a validation outside the wizard (e.g. the “Generate” button could be disabled with a tooltip “Please enter a job title first”).

Edge Cases & Error Handling

- **User Skips All Wizard Questions:** If the user simply clicks through without providing any input (e.g. enters nothing and hits “Skip” or just closes the wizard), the fallback template list is presented. This way, they are still given help rather than leaving them with an empty state. We essentially never just do nothing – either the wizard yields a

bullet, or the templates step in.

- **Wizard Input Too Long or Unwieldy:** If the user types a very long description in the wizard (e.g. pasting a whole paragraph of what they did), the AI prompt will still take it, but there's a risk the bullet comes out verbose. We will mitigate this by guiding the user in the UI to keep answers brief (perhaps a character limit or a note "1-2 short phrases"). In testing, concise inputs led to better bullets. If a long input is given, the AI will attempt to distill it, but we'll also rely on the "Improve" function to tighten any overly long bullet after generation.
- **LLM Output Quality Issues:** If the AI returns a bullet that is clearly off-base or nonsensical (despite our context, LLMs can still err), we handle it gracefully:
 - The system could display a message: *"This suggestion isn't a perfect fit? Try adjusting your inputs or regenerate."* with a quick **"Regenerate"** button. One idea: if the user clicks a thumbs-down on the suggestion (we could have feedback icons), it could automatically re-open the wizard or ask one more clarifying question to refine the context. For now, a simple regenerate button next to the bullet might suffice, calling the prompt again (possibly with a slight variation or an additional hint like "The previous attempt missed the mark; focus more on the technical aspect" if we have such feedback).
 - **No Overwrite without Consent:** If the user already had content in that bullet and used "Generate" (less likely, as quick-generate is more for empty bullets), we will confirm before replacing. But typically, "Tạo nhanh" is used on a new bullet slot or an empty entry.
- **Multiple Generated Bullets Similarity:** If a user uses the wizard repeatedly for the same role, there's a chance the outputs could sound similar ("Led X... Improved Y..." repeatedly). To prevent a monotonous tone, the AI needs to vary its phrasing if it sees similar patterns. Also, we might internally track the verbs used in existing bullets and instruct the AI to choose a different synonym for variety on subsequent calls. For example, first bullet used "Led", second might use "Implemented" or "Spearheaded". This ensures a more compelling and varied description list. It's a subtle enhancement that can be added to the prompt: *"Use a different action verb than those already used above."*
- **APIs and Failures:** In case of an API failure or network error during generation, the user will see a graceful error message instead of an empty bullet. e.g., *"Sorry, OkBuddy AI is taking a break. Please try again in a moment."*. The wizard can offer a "Retry" button in that state, or allow the user to cancel and return later. We ensure the user's inputs in the wizard aren't lost so they can just hit Generate again after a retry.

- **Template Library Growth:** If the job title is recognized (say from an internal database of common roles), we might tailor the fallback templates or example bullets to that role. For instance, after the wizard, or if skipped, we could say *“Here are examples for a Marketing Manager:”* and list bullets relevant to marketing management . This is an extension where OkBuddy could incorporate its Smart Library of example bullet points by title. It complements the AI generation by giving proven examples. (This was noted in our design as the “OkBuddy Smart Internal Library” of sample bullets). These could appear in the wizard as well (e.g. an option “Browse examples”).
- **Internationalization:** Currently, assume English output since the product spec is English. If the user’s resume is in another language, the prompts would need to detect that and either warn or attempt a translation. That’s outside current scope, but we’ll restrict to English for now (the UI text “Tạo nhanh” was Vietnamese, but presumably content is desired in English – we’ll clarify language settings in system settings elsewhere).
- **User Already Has Some Bullets:** If a user has manually written one bullet and uses the wizard to add another, the system handles it seamlessly by adding a new bullet field below the existing one. We ensure the insertion point is correct. Also, the “Improve all bullets” feature (if any) will include the new ones.
- **Undo:** We plan to allow undoing AI actions. For example, after a bullet is generated or improved, if the user doesn’t like it, they can Ctrl+Z or click an “Undo” toast to revert to the previous state (either empty or the original text) . This gives confidence to try AI features without fear of losing content.

Interaction & UI Behavior

- **Wizard UI Details:** The wizard is a modal dialog centered on the screen (or a popover anchored to the “Tạo nhanh” button). It uses a step indicator (e.g., “1 of 3”) or progress bar to show progress through the questions, since there are a few steps but it’s quick. Each step’s question is phrased in a friendly, encouraging way (matching OkBuddy’s tone). There are input validations on each (e.g., if the user enters extremely long text or leaves required ones blank, we highlight it).
- **Buttons and States:** The “Next” button on steps 1 and 2 is disabled until the user types at least something (we want at least a short phrase). A **“Skip”** link or secondary button is present on optional steps (and maybe on step 2 if they truly have no answer, though we’d prefer they attempt something). The **“Generate”** (final step) button triggers the API call. During generation, this button shows a loading spinner and is disabled to prevent double submissions. The modal might display a subtle animation (like an ellipsis “Generating...”) to reassure the user.

- **Modal Dismissal:** When the wizard closes (after generation or cancel), focus returns to the Work Experience editor. If a bullet was generated, it scrolls into view (if off-screen) and perhaps flashes or has a highlight to draw the user's eye to the new content, giving a sense of accomplishment.
- **Inline Editing of Generated Bullet:** The generated bullet is inserted as normal text in the bullet field. The user can click into it and edit like any other text. The system could initially render it in a **suggestion style (light yellow background)** to indicate "this is AI-generated, please review." Once the user clicks or types in it, the highlight disappears (implying the user has accepted or is modifying it).
- **Template Insert Interaction:** If the user chooses a bullet template, the placeholders within the text field could be actual input fields or tokens. We might implement them with a light gray background and italic text. Pressing the Tab key could jump to the next placeholder for quick keyboard navigation. This makes it easy to fill out the template without using the mouse to select each part.
- **Autocomplete Dropdowns:** As part of smart autocomplete, if the user types the first word and pauses, an autocomplete may appear. For example, typing "Led" could trigger a dropdown of common continuations like "a team of [#] to [achieve X]". This is similar to Google's Smart Compose but in a resume context. The suggestions appear in-line (grayed text) or as a dropdown list. The user can hit Tab or right-arrow to accept a suggestion. We will ensure these suggestions are contextually relevant, possibly informed by the job title or industry. (This feature exists generally in our editor for sentences , and we will extend it to bullet fields).
- **"Improve with AI" (Cải thiện) CTA Button.** When clicked:
 - The bullet field is temporarily disabled (read-only) while the AI rewrites it. We might show a spinner overlay on that bullet or change the wand icon to a loading animation.
 - After 1-2 seconds, the text is replaced with the improved version. We highlight the changes or the whole bullet briefly. If the improvement included adding a metric or keyword, those might be underlined to show what changed.
 - The user can undo or even see the original text via a tooltip or version toggle (perhaps show "Original | Improved" if they hover, similar to how Resume-Now shows original vs AI-generated text). However, at minimum an undo is provided.
- **Improve All (Bulk Optimization):** As an advanced feature, the Work Experience section's AI menu could offer **"Improve All Bullets"**. This will run the improve prompt on each bullet in that job entry sequentially . Each bullet field will be updated and highlighted. A single undo could revert all if needed. This is useful when the user finishes

writing or importing all bullets and wants to polish them in one go.

- **Real-Time Preview Update:** Throughout these interactions, the resume preview on the right side updates in real time. When the user is in the wizard, the preview remains unchanged (since we haven't added content yet). As soon as a bullet is generated or a template filled in, the preview will show that bullet under the job entry. This immediate feedback reinforces that something substantial has been accomplished – the resume is filling up with relevant content. It also lets the user check formatting and length (e.g., does the bullet wrap to two lines on the template, etc.).
- **Guidance and Tooltips:** We will incorporate small help texts where needed. For example, the wizard fields might have a tooltip or “Why ask this?” explanation: “We ask for a project and result so the AI can write a specific, impressive bullet point highlighting your achievement.” This can increase user buy-in to provide info, knowing the purpose. Similarly, the templates might have a top note: “**Tip:** Hiring managers love to see what you did and what impact it had. Fill in the blanks with your details, then let us polish the sentence!”
- **Button States:** The “Tạo nhanh” button itself might change to an “Add another” state after first use, as mentioned. Alternatively, it remains the same but always invokes the wizard anew. We ensure the label or icon clearly indicates AI assistance (perhaps a wand or magic icon next to “Generate bullet” text).
- **Mobile Responsiveness:** On smaller screens, the wizard could take full screen but still show step-by-step questions. The interaction remains mostly the same, but instead of a popover it might be a slide-in panel. The templates fallback could be a dropdown that expands to a list of options that are tap-able.
- **Visual Design:** We keep the wizard visually lightweight – maybe a simple illustration or icon at the top to make it friendly (a wizard hat or magic wand icon to signify “AI Assistant”). The inputs use the same styling as other text fields in the editor for consistency. The template placeholders might use a slight italic style and lighter color to denote “fill me”. Consistency with the overall Guided Editing design is maintained.
- **No Disruption of Manual Flow:** It's important that this new flow feels like a helpful addition, not a forced path. Users can still manually add a blank bullet and type from scratch if they prefer (the system might still offer ghost text suggestions or examples, but it won't force the wizard). The AI features are there to aid, not impede, the expert user who may already know what to write. Thus, all AI suggestion buttons are **opt-in**.

Summary Section – AI-Assisted Content Generation

Problem Statement

Users often struggle to write the Summary section of their resume, and feedback from concept tests indicates that AI-generated summaries can feel too generic or not aligned with the user's unique profile. The current quick generation for Summary may produce a bland paragraph that doesn't effectively highlight the user's strengths or match the job they're targeting. The problem is twofold: **users face blank-page anxiety with the Summary, and the AI's output isn't personalized enough**, leading to dissatisfaction.

Root Cause Analysis

- **Limited Data Utilization:** In the initial concept, the summary generator might not have fully leveraged the user's own work experience details or target job info, especially if used early. This leads to generic summaries (e.g., "Professional with proven track record in industry..."). The AI wasn't always grounding its output in the user's actual accomplishments or roles because that data was sparse or not fed in.
- **Generic Tone and Content:** Without context, the LLM resorts to safe, boilerplate language. Users sense that these lines could describe anyone, and thus feel the summary "doesn't sound like me." This generic tone fails to inspire users' confidence.
- **Sequence of Use:** Many users encounter the Summary as the first section, and if they generate it before filling other sections, the AI has almost nothing personal to go on. The empty-state summary generation becomes a shot in the dark. The guided editing flow intended for Summary to be done last, but not all users followed that, resulting in poor outputs.
- **Lack of User Guidance:** Users weren't prompted to specify what they wanted in their summary. If a user has particular career goals or key skills they want to emphasize, the current AI flow doesn't ask for that, so it might omit something the user cares about. This can make the AI summary seem off-target.

Solution Overview

To create a high-fidelity, personalized Summary, we combine automated generation with context from the user's profile and interactive refinement:

1. **Contextual AI Generation:** When the user clicks "Generate Summary with AI", the system will gather all relevant user data – work experience bullet points, job titles, years of experience, skills, and any target job description – to feed into the prompt. The AI will produce a first draft that is tailored to the user's background and the role they seek. This is not a blind generation; it's anchored in the content the user has provided elsewhere,

making it immediately more specific.

2. **Skeleton & Examples (Blank State Fallback):** If the user's profile is mostly empty (or they prefer starting manually), we provide a **summary skeleton template or example prompts** to guide writing. For instance, an empty Summary field might show placeholder text like: *"e.g. Passionate [Your Profession] with X years of experience in [Key Skill], known for [Key Achievement]..."* as a Mad-Libs style hint . Additionally, we can show a couple of **example summary statements** based on common roles (e.g., if the user's field is detected as Marketing, an example: "Marketing specialist with 5+ years of experience driving brand growth through data-driven campaigns...") . These give users inspiration and a starting structure if they don't want to auto-generate immediately.
3. **"Cải thiện với OkBuddy AI" Improvement:** Similar to Work Experience, the Summary section has an "Improve with AI" one-click option. If the user has written a draft (either manually or from the generator), they can click this to have the AI refine the summary. The improvement prompt will consider the user's existing summary text, plus cross-reference it with their work experiences and target job, to enhance clarity, impact, and keyword alignment. Essentially, it optimizes the summary, ensuring it's well-written and contains role-relevant highlights.
4. **Interactive Refinement (Edge Case Q&A):** In cases where the user hasn't provided enough info for a good summary, the system can ask quick questions (in a lightweight way, not a full wizard, to avoid redundancy). For example, if the user has no work experience filled and hits "Generate Summary," we might ask: *"What is your current profession or career goal?"* and *"Name one key strength or skill you'd like in your summary."* Using these two bits of input, the AI can then craft a more targeted summary. This is a fallback interactive step only if needed (since ideally, summary gen is done after other sections).

New UX Flow (Step-by-Step)

1. **Entry Point – Generate Summary:** In the Summary section card (which is at the top of the resume beneath Contact Info), if it's empty, a prominent CTA appears: **"Generate Summary with AI"** (possibly as a button in the empty text area or below the field) . Users can click this to invoke the AI generation. If the summary already has text (say the user wrote something or it was generated before), this button is replaced by the "Improve" icon for refinement.
2. **Data Gathering Behind the Scenes:** Once clicked, the system compiles the user's data to feed the AI:
 - All **Work Experience bullet points** the user has (or at least a summarized form: job titles, companies, and key achievements). For example: *"Software Engineer*

at ABC Corp (2019–2021): Led development of a CRM system improving data accuracy by 30%. Java, Python.” (We might combine the title and one key bullet if available).

- The user’s **total years of experience or seniority level**. If we can calculate (e.g., sum of job durations or a field where user entered years experience), we include that (e.g., “5 years of experience in software development”).
- **Key skills** from the Skills section (if the user filled any) or inferred from their bullets. If the user has a Skills section listing Python, Project Management, etc., we pass some of those as important keywords.
- If the user has entered a **Target Job Description** (through our job targeting feature), we extract a few key skills/keywords that the summary should mention (e.g., if target job emphasizes “cloud computing” and the user has that background).
- If Education or Certifications are significant (say the user is a recent grad with limited experience), we might include a note like “MSc in Computer Science”.
- *All this is done automatically; there’s no UI here, but it’s crucial to the prompt.* Essentially, the system acts as if it’s writing a brief bio of the user from available data, to inform the AI.

3. **AI Generation of Draft:** The LLM is prompted with an instruction to **draft a professional summary** paragraph for the user. The instruction might read: *“Write a 3-4 sentence professional summary for a resume using the following information about the candidate. Highlight their experience, skills, and accomplishments, and tailor it towards the target job when applicable. Tone: confident, concise, third-person (implied first-person), no pronouns like ‘I.’”* Then we inject the compiled data, for example:

- *Experience:* 5 years in software development; Software Engineer at ABC Corp – led CRM system project improving data accuracy 30%; Senior Developer at XYZ Inc – optimized payment system latency 20%.
- *Skills:* Java, Python, Project Management, AWS.
- *Education/Cert (if any):* B.Sc. in Computer Science.
- *Target Role:* Backend Engineer role requiring cloud computing, team leadership.

The model then generates a summary such as: *“Software Engineer with 5 years of experience in full-stack development. Proven success in leading projects that improve system performance and data accuracy (achieved 30% data accuracy*

improvement on a CRM project). Proficient in Java, Python, and AWS cloud technologies. Seeking to leverage strong project management and technical skills in a Backend Engineer role to drive innovative solutions.”

This draft is returned to the UI.

4. **Injection & Preview:** The generated summary text is placed into the Summary textarea field in the editor. We display it with some indication that it's AI-generated (e.g., highlight or a small note “AI draft” perhaps). The right-side resume preview updates immediately to show the summary under the header, giving the user a view of how it looks in context (font size, length, etc.). If the summary is too long (e.g., more than ~4 sentences or whatever the template supports without overflow), the user will see it spanning too much and can edit down. We also show a **word count** or character count below the field (common for summary section) – e.g., “75/100 words” – to guide them not to exceed recommended length .
5. **User Review & Edit:** The user reads the generated summary. Because it was tailored using their data, it should resonate more than a generic blurb. The user can directly click into the text and make any edits. Maybe they want to add another skill, change a wording, or remove something that's not relevant. The interface encourages this as an editable text area (the AI text is just a starting point).
6. **Interactive Suggestions (if no data):** If the user's profile was mostly empty and we couldn't gather much, the system might interject a quick Q&A before step 3. For example, after clicking “Generate Summary,” if we detect no work experiences or skills, we prompt: *“To create your summary, we need a bit of info. What's your current profession or a role you aim for?”* (Textbox) and then *“Name one or two strengths or skills you want to highlight.”* (Textbox). This could be a small popup or inline form. After the user provides these, we then feed those answers into the generation prompt (in lieu of work experience data). The rest of the flow proceeds with the AI draft. This ensures even a complete newbie user with a blank resume gets a somewhat personalized summary (e.g., “Aspiring Marketing Coordinator with strong skills in content creation and social media strategy...” if that's what they input).
7. **Example Library (alternative start):** Another path for blank summary: The UI could offer *“Not sure what to write? See examples.”* If clicked, a small list of 2-3 example summaries appears, possibly filtered by industry if known (or generic ones if not). User can copy one or insert it. For instance:
 - *“Detail-oriented **Accountant** with 4+ years of experience in financial analysis and reporting, adept at using Excel and SAP to streamline processes and ensure compliance.”*

- *“Creative **Graphic Designer** skilled in Adobe Creative Suite, with a track record of delivering compelling visual campaigns and brand identity projects for diverse clients.”*

These examples give users something to work with if they prefer not to auto-generate. They can choose one and edit it to fit their own details. This is analogous to the bullet templates but for the summary. (We have a repository of example summaries by field as part of content library) .

8. **“Improve Summary” CTA:** If the user has an existing summary (either generated as above, copied from example, or written from scratch), they can use the **“Cải thiện với OkBuddy AI”** on it at any time:
 - The **Improve icon** (magic wand) for the Summary is usually placed next to the summary field’s label or as a floating button when the field is focused . The user clicks it, triggering an AI rewrite of the entire summary text.
 - The prompt for improvement will send the current summary along with context (again, feeding in the user’s work experience and any job target info) and instruct the AI to **polish the writing**: improve grammar, make it more concise or impactful, incorporate any missing keywords (especially those found in the target job description that aren’t yet in the summary) , and possibly ensure a strong opening line.
 - For example, if the user’s draft said “I have worked on many projects in software development and am looking for new challenges,” the AI might return “Software developer with a history of leading successful projects, eager to tackle new challenges in fast-paced environments.” – thus removing first-person, adding a bit of oomph.
 - We handle the result similar to bullet improvement: the text is replaced in the field, highlighted for changes. We might also show suggestions inline: e.g., if adding a missing keyword, highlight it or add a comment like *“added ‘project management’”* so the user knows. If the summary was near the word limit, the AI will try to keep it trim (maybe by cutting weaker phrases).
 - The user can accept the improved summary or undo it if they preferred the original. They can also click improve again to iterate. Each click might yield a slightly different wording or incorporate any new manual edits the user made in between.
9. **Keyword Check & Suggestions:** As part of AI assistance, after generation or improvement, the system can analyze the summary for important keywords. If a target job is set, we compare the summary against the job’s top keywords (this is an existing

OkBuddy feature in concept). If some are missing, a subtle suggestion might appear: *“Consider mentioning in your summary.”* This could be a non-intrusive prompt below the field or an icon. The user can choose to accept it, which would trigger the AI to intelligently insert that keyword into the summary (either by adding a phrase or altering a sentence). For example, if the job ad stresses “project management” and the summary lacks it, clicking “Add ‘project management’” could result in the AI rewriting a sentence to: “...skilled in Java, Python, and project management.” This happens instantly, with the new text highlighted.

10. **Real-Time Typing Aid:** Similar to work experience bullets, while the user is manually typing in the summary field, the **smart autocomplete** can kick in. If the user pauses after typing a few words, the AI might suggest a continuation. E.g., user starts “Experienced project manager with expertise in”, the AI could suggest “ agile methodologies and cross-functional team leadership.” If the suggestion aligns, the user hits Tab to accept and save some typing. This transforms the summary writing into a assisted experience even without pressing a generate button.
11. **Saving and Persistence:** All these AI actions (generation, improvement, inserts) trigger autosave of the resume. The user’s summary won’t be lost. If they generate a summary and then navigate away, it’s saved just like any manual input.

Fallback & Edge Case Behaviors

- **Empty Profile Generation:** The biggest edge case is a user with a blank resume trying to generate a summary. As described, we handle this by either prompting for minimal info or showing examples. We do **not** want to generate a completely generic paragraph like “Dedicated professional seeking to leverage skills...” because that’s no better than a template. If the user declines to input anything even when prompted (e.g., they skip the “profession” question), we will either:
 - Generate a *very general* summary but make it clear it’s a placeholder (and encourage them to edit). For example, “Motivated professional with a passion for success and a drive to learn new skills.” This is obviously generic, but at least it fills the space. We’d accompany it with a note like *“Please edit this to reflect your details.”*
 - Or simply leave the summary blank and encourage them to fill work experience first. In fact, our guided approach in onboarding suggests doing Work Experience before Summary. The system could even detect this scenario and show a one-time tip: *“For best results, add a work experience or two, then come back to generate a tailored summary.”* (In the UI, maybe a tooltip on the disabled generate button or a banner).

- **Overwriting Existing Summary:** If the user already wrote a summary manually and clicks “Generate Summary,” we should confirm if we will overwrite. Likely in this case, the user should use “Improve” instead to refine what they have. We might hide the “Generate” button if text exists to avoid accidental overwrite. But if we do allow it (maybe via a submenu), we’d warn: *“This will replace your current summary with a new AI-generated one. Continue?”* to avoid surprise loss of user-written content.
- **Long Summaries:** We enforce brevity. If the AI returns a summary longer than our recommended length (say > 4 sentences or > ~120 words), we will truncate or instruct the AI with a stricter token limit. The UI also guides the user to stay within limits (word count indicator turns red if too long, etc.). If a user really wants a longer summary, they can still type it, but we inform them of best practice. Our improvement function also tries to trim unnecessary words (e.g., remove self-congratulatory fluff or repetitive phrases).
- **Tone Adjustments:** On the off chance the AI summary is too flat or too flamboyant, the user can prompt another try. We might offer preset **tone choices** after generation: like buttons for “Make it more formal” or “Make it more enthusiastic” if feedback shows users desire that control. This is an enhancement – currently we default to a neutral professional tone. But being aware of tone is important; e.g., exclamation points or subjective adjectives are generally avoided. The prompt is tuned accordingly.
- **Missing Target Job:** If no target job description is provided by the user, the summary generation simply won’t include that context. It will focus on the user’s background. This is fine – the output will be a general professional summary of the user. If a target job is later added, we could prompt the user like *“Update summary for target job?”* which on click would run the improve with the new context (inserting some keywords relevant to that job). So the system is adaptive: it can retrofit the summary to a job target whenever one is introduced.
- **Multiple Iterations:** The user can regenerate the summary if they don’t like the first draft. Perhaps we offer “**Regenerate**” button next to the result. Each time, we might vary the phrasing or order of sentences. The user can pick the best one or merge them. We will caution not to overdo it (because too many choices can be overwhelming), but 2-3 tries are available. The regenerate function will use the same prompt info; if truly different content is needed, we’d advise the user to edit their inputs (e.g., maybe add a missing skill to their profile, then regenerate to see it included).
- **International or Multi-language names:** If the user’s data includes non-English terms (say a company name or university that the AI might not recognize), the model might stumble. However, since we mostly feed these as-is and not ask it to translate or explain them, it should usually just incorporate them. We’ll monitor outputs for any garbling of proper nouns.

- **First-person in Input:** Sometimes users draft summaries in first person (“I am a software engineer...”) but resumes are conventionally written in third person implied (no “I”). Our AI improvements will remove first-person pronouns and adjust verbs accordingly. If a user writes “I managed a team of 5”, the improved might be “Managed a team of 5”. We explicitly handle this in the prompt instructions. This ensures consistency and adherence to resume norms.
- **Summary in Vietnamese or other language:** If the user’s resume is meant to be in Vietnamese (given the UI label “Tạo nhanh” was Vietnamese), how do we handle summary language? Ideally, we detect the preferred language and either use a Vietnamese model or translate prompts. For now, assume English output as default. As an edge case, if a user inputs profile data in Vietnamese and hits generate, the English LLM might produce an English summary (which would be mismatched). This could be addressed by language detection – but that might be beyond current scope. We note this as a future enhancement: multi-language support for AI generation.

Interaction & UI Behavior

- **Summary Section UI:** The summary is a single large textarea at the top of the resume editor. By default it might show a placeholder text in gray (the skeleton example text: *“e.g. Passionate Product Manager with 8 years of experience...”* to illustrate the format) . This placeholder disappears once the user clicks into the field or generates content.
- **Generate Summary Button:** When empty, a centered button or overlay text “✍️ Let OkBuddy write a draft for you” is shown . Clicking it triggers the data gathering and generation. While generating (which might take 2-3 seconds), we show a loading state: perhaps the button turns into “Generating...” or a spinner appears in the text area. We might also use a fun tagline like “OkBuddy AI is composing your summary...”.
- **Summary Generated State:** The moment text is inserted, if it’s longer than one line, the textarea auto-expands to fit it (or we remove the fixed height). The user can scroll within it if needed, but usually summaries are short enough. We highlight the text background just as with bullets to denote an AI suggestion. Perhaps we also show a small “**Edit** ✎” **prompt** or tooltip on it to nudge the user that they can and should edit any part they don’t love – essentially encouraging them to personalize further, even though AI wrote it.
- **Examples Dropdown:** If the user clicks “See examples” (if we implement this), a small popup near the field will list a few sample summaries. Each might have a “Insert” button next to it. If inserted, it just populates the field with that text (treated as if user typed it, but we could still highlight it as suggestion). The user then edits and uses improve if desired. This examples UI will likely be context-aware: if we know the user’s job title from work experience or desired role (say they filled in a Headline or something), we can filter examples of that field. Otherwise, it might list a generic example and one for a common

field like “Software Engineer” and “Marketing Specialist” to cover two extremes.

- **Improve Summary Interaction:** The improve button (magic wand) for summary likely sits to the top-right of the summary field or next to the “Summary” label. It might be consistently placed where other section actions are. On hover it might say “Improve summary with AI”. When clicked, similar to bullet improve, the summary field is momentarily locked and possibly the text color changes to indicate processing. We then either replace the text directly, or we could do a side-by-side diff. A simple approach: replace in place, highlight changes (maybe green for added words, red strikethrough for removed, shown temporarily). If the changes are extensive, an alternative is to show the new version in a modal comparing old/new. But that might be overkill; inline update with undo is sufficient for now.
- **Undo and History:** We ensure the user can undo any AI change. A common pattern is showing a snackbar “ Summary improved. [Undo]” after the operation. Clicking undo reverts to the previous text. This is important if the AI does something the user doesn’t like. We’ll keep at most one level of undo for AI operations (for more, user can always regenerate/improve again).
- **Real-time Feedback:** As the user edits the summary (manually or after generation), the preview updates each keystroke (or on field blur for performance). So the user can see exactly how the summary looks in the resume layout (maybe it’s in a smaller font in the preview, etc.). If their edit makes it too long and flows onto a second line in the preview’s design, they’ll notice and possibly trim it. We also provide possibly a **character/word limit indicator** beneath the field (e.g., “85/100 words”). If the template has a hard limit (some designs only allocate a certain space), we reflect that.
- **Guidance Text:** Under the summary field, we display helpful guidance, such as: “Your summary should be 2-4 sentences highlighting your biggest strengths and experiences. Keep it concise and relevant to the job you want.” This static tip is always visible to set expectations. If the user hasn’t used the generator yet, this may suffice to encourage them to try it or write something.
- **Target Job Integration:** If a target job is set in the system, we may show a small label or icon in the summary section header (e.g., a  icon) indicating that suggestions are tailored to that target. If the user clicks it (or an info icon), it could say “Using target job: [Job Title] at [Company] to customize suggestions”. This reinforces that the AI is incorporating that context (which adds transparency to why certain keywords appear in the AI output).
- **After Generation – User Path:** We anticipate a common path: user fills work experience via AI bullets, then clicks generate summary, gets a decent draft, uses improve once, and is done. Another path: user might skip generating and write a rough summary themselves, then click improve to polish. We support both. The UI elements (Generate

vs Improve) adapt based on whether the field is empty. If not empty, “Generate Summary” might be hidden to avoid confusion (we don’t want two different AI actions visible; that’s why we limit one primary CTA at a time).

- **Mobile UX:** On mobile, the summary field might be full-screen when focused. The “Generate” button might be a large button below the field or an overlay prompt. The improve icon might float or be in a toolbar above the keyboard. We ensure that the suggestions (like example dropdown or keyword suggestions) are accessible in a small screen format (they might appear as an action sheet from bottom).
- **Visual Consistency:** We match the style of the rest of the guided editor. For example, section headers (Summary) likely have some icon or consistent placement for AI functions (like how Work Experience had the AI dropdown on header in the concept) . Possibly the summary section header has an AI menu that includes “Generate Summary” and “Improve Summary” if we want to consolidate. In our spec, we present them as separate context-aware buttons for simplicity. But engineers can decide to have one icon that opens a menu: if empty, menu shows “Generate draft”; if not, shows “Improve” and maybe “Re-generate fresh” etc.
- **Performance Considerations:** The summary generation uses more context (all work ex bullets, etc.), which could be a larger prompt for the LLM. We may use a moderately high token limit model. We will ensure the request is optimized (perhaps compress bullet points to one line each or only feed the most relevant ones if too many). This should be fine for typical users (e.g. 3-5 jobs with 3 bullets each is manageable input size).
- **User Education and Trust:** We might include a one-time tooltip or onboarding hint when the user first visits the Summary section (especially if it’s empty). Something like: “**Tip:** Fill in your Work Experience first, then let OkBuddy’s AI draft a summary highlighting your achievements.” This guides the user to the ideal usage sequence. We can detect if they have work experience filled; if yes, show the generate button brightly. If not, the generate button might be grayed out or accompanied by a note “For best results, add a work experience or two below first.”

AI Prompt Logic for Summary Generation & Improvement

The intelligence of the Summary assistant lies in how we craft the AI prompts using user data:

- **Summary Generation Prompt:** We feed the compiled user info into the LLM with a clear instruction. For example:

You are an AI resume assistant. Write a concise 3-4 sentence professional summary for the user's resume based on the following information:

- Roles & Achievements: Product Manager (2018–2021) – led launch of 3 products, increasing market share by 15%. Marketing Coordinator (2015–2018) – improved social media engagement by 50%.
- Skills: Product Strategy, Data Analysis, Cross-functional Leadership.
- Education: B.A. in Business Administration.
- Target Job: Senior Product Manager (requires agile project management, leadership, data-driven decision making).

The summary should highlight the user's years of experience, key skills, and significant accomplishments, and align with the target job. Use third-person (no "I") and a confident tone.

- The model will then produce something akin to: *“Product Manager with 7+ years of experience leading cross-functional teams to launch successful products. Proven ability to drive market share growth (15% increase) and boost engagement through data-driven strategies. Skilled in product strategy, data analysis, and agile project management. Seeking to leverage leadership and innovation in a Senior Product Manager role.”*

Note how it integrated the metrics and keywords from the input. By structuring the input as bullet points of facts, we help the model focus on specifics rather than filler.

- **Inclusion/Exclusion Logic:** If some element is missing:
 1. No target job given: The prompt will omit the target job line and simply say “Write a summary based on the following info” with their roles, skills, etc. The AI will then produce a more generalized summary of the candidate. It might end with a generic statement like “seeking new opportunities” or just stop after listing strengths.
 2. If skills are missing, we don’t include that line. The AI might then focus more on roles and general strengths (“dynamic professional” etc.).
 3. If work experience is missing (like a student user), we prompt based on education and any projects/skills. E.g., “Computer Science graduate with strong skills in X... seeking to ...”. We explicitly handle this case by adjusting the prompt: e.g., “based on the following education and skills” instead of roles.
- **Summary Improvement Prompt:** When improving an existing summary, the prompt includes:
 1. The user’s **current summary text** (so the AI knows what it’s starting from).
 2. Potentially a brief recap of profile data again (to reinforce any keywords or achievements that could be added if missing). If the current summary is already pretty complete, feeding all data might not be necessary, but it can help identify

gaps.

3. Instructions to **refine style and content**: e.g., “Polish the following summary to be more concise and impactful. Ensure it’s in third person, incorporates relevant skills or results from the profile, and aligns with the target job description. Suggest improvements subtly, without changing the user’s professional story.”
 4. This way, if the original summary omitted a big achievement from the work experience, the AI might cleverly insert it: maybe adding a clause “with a track record of improving data accuracy by 30%” if that was in their bullets but not mentioned. Or if the target job needs “Python” and the summary didn’t mention it, the improved version will slip in “skilled in Java and Python” if appropriate.
 5. The output is essentially a tweaked version of their summary. We expect improvements like: removing any first-person usage, correcting any grammar, breaking up run-on sentences, adding a metric or keyword, and possibly reordering for emphasis (often leading with years of experience or role title for punch).
- **Use of Job Description**: Both generation and improvement prompts will leverage the target job description if present. We might not feed the entire text of the JD (too long), but rather a parsed list of the top 5-10 keywords/skills from it (which our system likely extracted for keyword suggestions feature). This ensures the AI knows what the user is aiming for. For example, if the JD repeatedly mentions “machine learning” and the user has ML experience, the summary should highlight that. If the user doesn’t have a certain required skill, obviously the AI won’t invent it, but it might emphasize willingness to learn or related skills if appropriate (we have to be careful not to add content the user didn’t provide).
 - **First-Person Cleanup**: We specifically program the prompt to remove “I” or “my” statements. If the user’s original summary was “I am looking for a challenging role...”, the AI output should be “Seeking a challenging role...” with no “I”. This is straightforward and our tests show the model handles it well when instructed.
 - **Optimizing Length**: The prompt can include a directive like “Total length ~50-80 words” to keep it in check. And “do not add extraneous filler”. This pushes the model to be concise. If the user’s input was too long, it will compress it.
 - **Tone and Person**: We maintain an **enthusiastic but professional tone**. Words like “passionate”, “driven”, “proven” are common and acceptable, but we avoid any language that’s overly casual or too salesy. The AI will typically output something neutral-professional given the instructions and examples.

- **Multiple Sentences Structure:** We aim for 2-4 sentences. Often a good pattern is:
 1. **Who the candidate is** (title or general role, years experience, domain).
 2. **Top skills/strengths** (especially ones relevant to the job).
 3. **Notable achievement or qualification** (maybe with a number or specific if available).
 4. **Goal or target** (what they are looking for, or a closing that aligns with the next role).

Our prompt will encourage this structure implicitly by the way we list info. We might also include in instructions: “The first sentence should introduce the candidate broadly, the second should give a key achievement, etc.” However, too rigid instructions might confuse the model; providing the info in bullet form as we plan often naturally yields a well-structured paragraph.

- **No Repetition:** If the user’s bullets already have some phrases that the summary generation might echo, we’ll try to vary it. For instance, if a bullet says “improved data accuracy by 30%,” and we feed that, the summary might reuse the phrase. That’s okay once, but if multiple bullets have metrics, the summary can’t list them all. The AI will likely pick one or two to mention. We instruct it to highlight *overall impact* rather than list every stat, to avoid sounding too granular.
- **Testing and Iteration:** We will test the prompt with a variety of user profiles to ensure the outputs are indeed better than the previous approach. The goal is a summary that **feels personal and complementary** to the rest of the resume. It should not just rehash every bullet, but rather synthesize them into a cohesive story – this is something we expect the GPT model to handle given the right input.

Edge Cases & Error Handling

- **AI Failure:** If the summary generation call fails (timeout or error), we show an inline error message in the summary field area: *“Oops, OkBuddy AI couldn’t generate the summary. Check your connection and try again.”* The user can retry with a button. We also ensure that if it fails after the user provided interactive inputs (in the empty profile Q&A case), we do not discard those inputs – we re-use them on retry so the user doesn’t have to re-type.
- **Partial Updates:** If the AI returns a result that is unusually formatted (say it returns bullet points or a first-person by mistake), we will clean it up in post-processing before displaying. For example, if it returned “- 5 years experience in X...” as a bullet list, we’d

remove the dash and join into sentences. These are unlikely with proper prompting, but we add guards.

- **HTML Injection:** Ensure the AI output is plain text (no markdown or weird characters). If it mentions the user's name (it shouldn't, as name is in contact info, not in summary), we remove that (some older AI patterns included the name; we explicitly avoid that).
- **Data Privacy:** All user data stays on our servers or is sent to the LLM via API securely – nothing is stored outside. This is more a backend note, but we ensure we comply with any privacy standards.
- **User Overrides:** If a user absolutely doesn't want AI involvement, they can ignore these buttons. We might allow them to disable AI suggestions, though that's probably not needed in UI explicitly (the user can just choose not to click). But all default behaviors (like placeholders or example suggestions) should not be too intrusive for those who prefer manual writing.
- **Autosave Conflict:** If the user starts typing while an AI generation is in progress (say they clicked generate but then immediately started typing impatiently), we handle it by either disabling input until generation is done, or if we allow it, we cancel the AI result to not override user typing. It's safer to disable input during generation to avoid that conflict.
- **Concurrent Edits:** If an improve operation is triggered while the user is editing the text, we either prevent it (require them to finish editing, maybe by losing focus) or take the latest text as of the moment of click. We document that behavior to avoid any confusion (e.g., if they had unsaved edits, though since we autosave keystrokes, it's always saved).

How This Solves Original Feedback

- **Personalization & Relevance:** By using the user's own data (experience bullets, skills, etc.) the generated summary is **specific to them**. This directly addresses the "generic AI output" complaint. For example, instead of a generic line like "Results-driven professional with many years of experience," the new approach produces something like "Software Engineer with 5 years of experience in fintech and cloud services" – instantly more relevant. Users in testing will recognize their history in the summary, making it credible and helpful.
- **Highlights Real Achievements:** Because the prompt can pull in a key achievement (e.g., "improved X by 30%"), the summary now highlights what makes the user stand out. This was likely missing before. The concept feedback wanted summaries that "match my real experience" – now they will, since literally derived from it. It essentially **summarizes the resume**, which is the intended purpose of a Summary, rather than

some canned opening line.

- **Reduces User Effort, Maintains Control:** Users still avoid the blank page. They get a draft in seconds that would have taken much longer to write from scratch. But importantly, the draft is more on-point, requiring fewer edits. If they do want changes, the improve button and direct editing are at their fingertips. So the user is in control of the final wording, but we've gotten them ~90% there. This combination of **speed + accuracy** is key to user satisfaction.
- **Incorporates Targeting Seamlessly:** A major benefit of OkBuddy is tailoring to job descriptions. The summary generator/improver now explicitly leverages the target job info (if provided) to inject relevant keywords. Users from concept tests who complained that the AI summary didn't feel targeted will see a difference – e.g., if the JD asks for “project management” and they have that skill, the summary will mention it. This makes the user feel the AI is actually helping them tailor their resume, not just generically reword it. It **addresses ATS optimization** in the summary, which can be a differentiator (many other tools focus AI on bullets only).
- **Guidance for Beginners:** For users who had no idea how to start a summary, the placeholders and examples give them direction. This **mirrors the wizard approach** but in a lighter-touch way, since a full wizard for summary might be overkill. Instead, a single-click with some behind-the-scenes work produces a decent paragraph. The user can learn from that structure. If they were truly lost, now they have something concrete to react to, which is psychologically much easier than inventing text from nothing.
- **Quality and Consistency:** The AI-improved summaries will be grammatically correct, well-structured, and consistent with a professional tone. Any awkward phrasing a user might have written can be smoothed out. This raises the overall quality of resumes produced. Users in testing will likely no longer comment that the summary feels “off” or “bland” – especially if they use the improve function on their own attempt. The summary will read like it was written by someone with resume expertise, but containing the **user's unique details**. That hits the sweet spot of what we want OkBuddy to deliver.
- **Time Savings and Confidence:** Writing a good summary can take a lot of time and second-guessing. With AI assistance, that time is cut dramatically. Users can get a strong draft in one click, and an even stronger final version in another click (Improve). This not only saves time but also gives them confidence that their summary is well-written. Knowing that it's optimized (with the right keywords, etc.) can reduce anxiety about “is my resume good enough?”. We foresee users being pleasantly surprised that the often dreaded summary section became one of the easiest, thanks to OkBuddy.
- **Feedback-Driven Adjustments:** If concept testing indicated certain phrases or styles users hated (for instance, some might dislike overly “buzzwordy” summaries), we can

fine-tune the prompt or even add a style toggle. Because our solution is flexible, we can adapt. But the core improvement is the grounding in user data, which inherently produces less buzzword fluff and more factual content.

- **End-to-End Coherence:** By generating the summary after the work experience, we ensure the summary aligns with what follows in the resume. It might even reference things implicitly that are detailed later. This **coherence** was lacking when summary was generated in isolation. Now it reads as a true summary of the resume. Users will notice that the resume “tells a consistent story” from summary to bullets, which makes the document stronger. It also means if they update work experience later, they might regenerate or improve the summary to keep it in sync – a supported use case.
- **Competitive Edge:** Competitors like Resume Genius explicitly mention using ChatGPT for summary generation, but if those outputs are generic, users will prefer OkBuddy’s approach that feels more tailored. We allow both automation and customization. This spec ensures that OkBuddy’s summary section is not just an AI gimmick, but a genuinely useful tool that produces better outcomes than a user would likely write on their own (especially those who aren’t confident writers). That leads to user delight and positive word-of-mouth.
- **Mitigating Common Mistakes:** Many users write summaries that are too long, too vague, or just re-list their duties. Our AI approach inherently avoids these pitfalls by design:
 - Too long? Our prompt keeps it short and the model usually abides by that.
 - Too vague? We feed specifics so the model will mention concrete skills or achievements, not empty adjectives.
 - Re-list duties? The model focuses on high-level strengths and results, not tiny details, making the summary additive rather than redundant relative to work experience.

This addresses implicit feedback where resumes often had weak summaries. We’re helping every user have a compelling introduction, which can greatly improve how a recruiter reads their resume (first impressions).

In conclusion, the Summary section AI spec ensures that OkBuddy provides a **personalized, high-impact summary generation and editing experience**. By leveraging context and interactive improvements, we solve the earlier issues of irrelevance and generic output. Users will get the dual benefit of **speed and quality** in crafting their professional summary, rounding out the resume building journey on a high note. With Work Experience bullets and Summary

both assisted by AI in a user-centered way, OkBuddy positions itself as a truly intelligent resume assistant that addresses users' real frustrations and helps them put their best foot forward.